

Complete Python Notes

Definitions + short examples • Beginner → Job-ready

How to use these notes: Learn the definition first, then type the example yourself. For developer interviews, focus especially on functions, OOP, exceptions, modules, iterators/generators, APIs, databases, testing, async Python, Git, and security.

1. Python Fundamentals

Python

Definition: Python is a high-level, general-purpose programming language known for readable syntax and a large standard library.

Example:

```
print('Hello, Python!')
```

Interpreter

Definition: An interpreter executes program instructions at runtime rather than producing a traditional standalone machine-code executable first.

Example:

```
python app.py
```

Source code

Definition: Source code is the human-readable program written by a programmer.

Example:

```
x = 10
```

Statement

Definition: A statement is an instruction that Python can execute.

Example:

```
x = 10
```

Expression

Definition: An expression is code that produces a value.

Example:

```
2 + 3
```

Indentation

Definition: Indentation is whitespace used by Python to define blocks of code.

Example:

```
if x > 0:  
    print('positive')
```

Comment

Definition: A comment is text ignored by the Python interpreter and used to explain code.

Example:

```
# calculate total
```

Identifier

Definition: An identifier is a name used for variables, functions, classes, modules, and other objects.

Example:

```
total_price = 500
```

Keyword

Definition: A keyword is a reserved word with a predefined meaning in Python.

Example:

```
if, else, for, class, return
```

2. Variables and Data Types

Variable

Definition: A variable is a name bound to an object or value.

Example:

```
age = 25
```

Object

Definition: An object is a value stored in Python's object model. Every Python object has an identity, type, and value.

Example:

```
x = [1, 2, 3]
```

Dynamic typing

Definition: Python determines an object's type at runtime; a variable name can later refer to an object of another type.

Example:

```
x = 10  
x = 'ten'
```

int

Definition: The int type represents integers.

Example:

```
count = 10
```

float

Definition: The float type represents floating-point numbers.

Example:

```
price = 19.99
```

complex

Definition: The complex type represents complex numbers with real and imaginary parts.

Example:

```
z = 2 + 3j
```

bool

Definition: The bool type represents truth values: True and False.

Example:

```
is_valid = True
```

str

Definition: The str type represents Unicode text.

Example:

```
name = 'Preeti'
```

None

Definition: None is Python's singleton object commonly used to represent absence of a value.

Example:

```
result = None
```

Mutable object

Definition: A mutable object can be changed after creation.

Example:

```
items = [1, 2]
items.append(3)
```

Immutable object

Definition: An immutable object cannot be changed after creation; operations create another object instead.

Example:

```
s = 'cat'
s = s + 's'
```

Type conversion

Definition: Type conversion changes a value from one type to another when supported.

Example:

```
age = int('25')
```

type()

Definition: type() returns the type of an object.

Example:

```
type(10) # int
```

isinstance()

Definition: isinstance() checks whether an object is an instance of a type or tuple of types.

Example:

```
isinstance(10, int) # True
```

3. Operators

Arithmetic operators

Definition: Operators used for mathematical calculations: +, -, *, /, //, %, and **.

Example:

```
7 // 2 # 3
```

Comparison operators

Definition: Operators that compare values and produce a Boolean result: ==, !=, <, <=, >, >=.

Example:

```
age >= 18
```

Logical operators

Definition: and, or, and not combine or invert Boolean expressions.

Example:

```
age >= 18 and citizen
```

Assignment operators

Definition: Operators such as =, +=, -=, *= and /= assign or update values.

Example:

```
x += 2
```

Identity operators

Definition: is and is not test whether two references point to the same object.

Example:

```
a is b
```

Membership operators

Definition: in and not in test membership in a container.

Example:

```
'a' in 'cat'
```

Operator precedence

Definition: Operator precedence determines the order in which parts of an expression are evaluated.

Example:

```
2 + 3 * 4 # 14
```

4. Strings

String indexing

Definition: Indexing accesses a character by position, starting at 0.

Example:

```
word = 'Python'  
word[0] # 'P'
```

String slicing

Definition: Slicing extracts a sequence using start, stop, and optional step.

Example:

```
word[0:3] # 'Pyt'
```

f-string

Definition: An f-string embeds expressions directly inside a string literal.

Example:

```
name = 'Asha'  
f'Hello {name}'
```

String method

Definition: A string method is a built-in operation available on string objects, such as `upper()`, `lower()`, `split()`, and `replace()`.

Example:

```
'hello'.upper()
```

String immutability

Definition: Strings cannot be modified in place; operations return new strings.

Example:

```
s = 'cat'
s = s.replace('c', 'b')
```

5. Collections

List

Definition: A list is an ordered, mutable collection that can contain duplicate values.

Example:

```
nums = [1, 2, 2, 3]
```

Tuple

Definition: A tuple is an ordered, immutable collection.

Example:

```
point = (10, 20)
```

Set

Definition: A set is an unordered collection of unique hashable elements.

Example:

```
tags = {'python', 'api'}
```

Dictionary

Definition: A dictionary is a mutable mapping of keys to values.

Example:

```
user = {'name': 'Asha', 'age': 25}
```

List comprehension

Definition: A compact syntax for creating lists from an iterable, optionally with filtering.

Example:

```
[x*x for x in range(5)]
```

Set comprehension

Definition: A compact syntax for creating sets.

Example:

```
{x*x for x in range(5)}
```

Dictionary comprehension

Definition: A compact syntax for creating dictionaries.

Example:

```
{x: x*x for x in range(5)}
```

Hashable

Definition: A hashable object has a stable hash value and can be used as a dictionary key or set element.

Example:

```
d = {(1, 2): 'point'}
```

6. Control Flow

if statement

Definition: Executes a block when a condition is true.

Example:

```
if score >= 50:  
    print('Pass')
```

elif

Definition: Checks another condition when previous if/elif conditions were false.

Example:

```
if x > 0:  
    ...  
elif x == 0:  
    ...
```

else

Definition: Provides a fallback block when preceding conditions are false.

Example:

```
if ok:  
    ...  
else:  
    ...
```

for loop

Definition: Iterates over items from an iterable.

Example:

```
for item in items:  
    print(item)
```

while loop

Definition: Repeats a block while a condition remains true.

Example:

```
while n > 0:  
    n -= 1
```

break

Definition: Immediately terminates the nearest loop.

Example:

```
for x in nums:
    if x == 5: break
```

continue

Definition: Skips the rest of the current loop iteration.

Example:

```
for x in nums:
    if x < 0: continue
```

pass

Definition: A no-operation statement used as a placeholder.

Example:

```
def future_feature():
    pass
```

match statement

Definition: Structural pattern matching selects behavior based on the shape or value of data.

Example:

```
match command:
    case 'start': ...
```

7. Functions

Function

Definition: A function is a reusable block of code that performs a task and can accept inputs and return a result.

Example:

```
def add(a, b):
    return a + b
```

Parameter

Definition: A parameter is a name in a function definition that receives an argument.

Example:

```
def greet(name): ...
```

Argument

Definition: An argument is a value supplied when calling a function.

Example:

```
greet('Asha')
```

Return value

Definition: A return value is the result sent back by a function using return.

Example:

```
return total
```

Default parameter

Definition: A parameter with a default value is used when the caller omits that argument.

Example:

```
def greet(name='Guest'): ...
```

*args

Definition: *args collects extra positional arguments into a tuple.

Example:

```
def total(*nums):  
    return sum(nums)
```

**kwargs

Definition: **kwargs collects extra keyword arguments into a dictionary.

Example:

```
def show(**data):  
    print(data)
```

Lambda

Definition: A lambda is a small anonymous function expressed as a single expression.

Example:

```
square = lambda x: x*x
```

Recursion

Definition: Recursion is when a function calls itself to solve smaller instances of a problem.

Example:

```
def fact(n):  
    return 1 if n == 0 else n * fact(n-1)
```

Docstring

Definition: A docstring is a string used to document a module, class, or function.

Example:

```
def add(a,b):  
    '''Return the sum.'''  
    return a+b
```

Scope

Definition: Scope defines where a name can be accessed. Python commonly follows LEGB: Local, Enclosing, Global, Built-in.

Example:

```
x = 'global'
```

8. Exceptions and Error Handling

Exception

Definition: An exception is an event raised during execution that can disrupt normal program flow.

Example:

```
1 / 0 # ZeroDivisionError
```

SyntaxError

Definition: A SyntaxError occurs when Python cannot parse the source code.

Example:

```
if True print('x')
```

try/except

Definition: try contains code that may raise an exception; except handles matching exceptions.

Example:

```
try:
    x = int(text)
except ValueError:
    x = 0
```

else in exception handling

Definition: The else block runs when the try block completes without an exception.

Example:

```
try:
    x = int(text)
except ValueError:
    ...
else:
    ...
```

finally

Definition: The finally block runs whether an exception occurs or not, commonly for cleanup.

Example:

```
try:
    f = open('a.txt')
finally:
    f.close()
```

raise

Definition: raise explicitly triggers an exception.

Example:

```
raise ValueError('Invalid age')
```

Custom exception

Definition: A custom exception is a user-defined exception class, usually derived from Exception.

Example:

```
class InvalidAgeError(Exception):
    pass
```

9. File Handling

File object

Definition: A file object represents an open file and provides methods for reading and writing.

Example:

```
with open('data.txt') as f:
    text = f.read()
```

Context manager

Definition: A context manager controls setup and cleanup around a block, commonly used with `with`.

Example:

```
with open('data.txt') as f:
    data = f.read()
```

with statement

Definition: The `with` statement enters a context manager and ensures its cleanup method is called.

Example:

```
with open('x.txt', 'w') as f:
    f.write('Hi')
```

JSON

Definition: JSON is a text-based data-interchange format commonly used by APIs.

Example:

```
import json
json.dumps({'id': 1})
```

10. Modules, Packages and Environments

Module

Definition: A module is a Python file containing definitions and executable code that can be imported.

Example:

```
import math
```

Package

Definition: A package is a structured collection of Python modules, typically represented by a directory.

Example:

```
from myapp import utils
```

import

Definition: `import` loads a module or selected names so they can be used in a program.

Example:

```
import os
```

pip

Definition: `pip` is the standard package installer commonly used to install Python packages from package indexes.

Example:

```
pip install requests
```

Virtual environment

Definition: A virtual environment isolates project dependencies from the system Python environment.

Example:

```
python -m venv .venv
```

requirements.txt

Definition: A requirements file lists project dependencies and often their versions.

Example:

```
requests==2.32.0
```

PYTHONPATH

Definition: PYTHONPATH is an environment variable that can add directories to Python's module search path.

Example:

```
PYTHONPATH=/project
```

11. Object-Oriented Programming

Class

Definition: A class is a blueprint for creating objects with attributes and behavior.

Example:

```
class User:
    pass
```

Instance

Definition: An instance is an object created from a class.

Example:

```
u = User()
```

Attribute

Definition: An attribute is data associated with an object or class.

Example:

```
u.name = 'Asha'
```

Method

Definition: A method is a function defined inside a class.

Example:

```
class User:
    def greet(self): ...
```

__init__

Definition: __init__ is an initializer method commonly used to set up instance attributes after object creation.

Example:

```
def __init__(self, name):
    self.name = name
```

self

Definition: self is the conventional name for the instance passed to an instance method.

Example:

```
def greet(self):
    return self.name
```

Encapsulation

Definition: Encapsulation organizes data and behavior together and can restrict or control access through an object's interface.

Example:

```
class Account:
    def __init__(self): self._balance = 0
```

Inheritance

Definition: Inheritance allows a class to derive behavior and attributes from another class.

Example:

```
class Dog(Animal):
    pass
```

Polymorphism

Definition: Polymorphism allows different objects to provide compatible behavior through a common interface.

Example:

```
for animal in animals:
    animal.speak()
```

Abstraction

Definition: Abstraction exposes essential behavior while hiding unnecessary implementation details.

Example:

```
class Payment:
    def pay(self): ...
```

Composition

Definition: Composition builds a class from objects of other classes rather than relying only on inheritance.

Example:

```
class Car:
    def __init__(self): self.engine = Engine()
```

@property

Definition: @property lets a method be accessed like an attribute and can control access or validation.

Example:

```
class User:
    @property
    def name(self): return self._name
```

Class method

Definition: A class method receives the class as its first argument and is declared with @classmethod.

Example:

```
class User:
    @classmethod
    def guest(cls): return cls()
```

Static method

Definition: A static method is a function placed in a class namespace that does not receive an implicit instance or class.

Example:

```
class Math:
    @staticmethod
    def add(a,b): return a+b
```

12. Iterators and Generators

Iterable

Definition: An iterable is an object that can return its elements one at a time, typically through `iter()`.

Example:

```
for x in [1,2,3]: print(x)
```

Iterator

Definition: An iterator produces successive values through `__next__()` and signals completion with `StopIteration`.

Example:

```
it = iter([1,2,3])
next(it)
```

Generator

Definition: A generator is a special iterator commonly created with a function containing `yield`; values are produced lazily.

Example:

```
def nums():
    yield 1
    yield 2
```

yield

Definition: `yield` pauses a generator function and produces a value while preserving its execution state.

Example:

```
yield item
```

Lazy evaluation

Definition: Lazy evaluation delays computation until the result is needed, often reducing memory use.

Example:

```
squares = (x*x for x in range(10**6))
```

13. Functional and Advanced Python

map()

Definition: `map()` applies a function to each item of an iterable and returns an iterator.

Example:

```
list(map(str, [1,2,3]))
```

filter()

Definition: filter() keeps items for which a predicate is true and returns an iterator.

Example:

```
list(filter(lambda x: x>0, nums))
```

reduce()

Definition: reduce() repeatedly combines iterable items into a single result; it is provided by functools.

Example:

```
from functools import reduce
```

Decorator

Definition: A decorator is a callable that wraps another function or class to extend or modify behavior.

Example:

```
def log(func):
def wrapper(*a, **k): return func(*a, **k)
return wrapper
```

@decorator syntax

Definition: The @name syntax applies a decorator to the following function or class.

Example:

```
@log
def work(): pass
```

Closure

Definition: A closure is a function that retains access to variables from its enclosing scope after that scope has finished.

Example:

```
def outer(x):
def inner(): return x
return inner
```

Higher-order function

Definition: A higher-order function accepts functions as arguments, returns functions, or both.

Example:

```
sorted(words, key=len)
```

Unpacking

Definition: Unpacking assigns elements from an iterable to multiple names.

Example:

```
a, b = [10, 20]
```

Walrus operator

Definition: The := operator assigns a value within an expression.

Example:

```
if (n := len(items)) > 0:
print(n)
```

14. Type Hints and Modern Python

Type hint

Definition: A type hint annotates expected types to improve readability and support static analysis; Python generally does not enforce it at runtime.

Example:

```
def add(a: int, b: int) -> int: ...
```

Optional type

Definition: Optional traditionally indicates a value may be of a type or None; modern Python can use T | None.

Example:

```
name: str | None = None
```

Generic

Definition: A generic type represents a family of related types parameterized by another type.

Example:

```
list[int]
```

dataclass

Definition: A dataclass reduces boilerplate for classes mainly used to store data.

Example:

```
@dataclass
class User:
    name: str
```

Enum

Definition: An Enum defines a fixed set of named symbolic values.

Example:

```
class Color(Enum):
    RED = 'red'
```

15. Testing, Logging and Debugging

Unit test

Definition: A unit test checks a small, isolated piece of code, such as a function.

Example:

```
def test_add():
    assert add(2,3) == 5
```

pytest

Definition: pytest is a popular Python testing framework with simple test discovery and assertion support.

Example:

```
pytest tests/
```

Assertion

Definition: An assertion checks that a condition is true and raises AssertionError if it is false.

Example:

```
assert total >= 0
```

Mock

Definition: A mock is a test double used to replace a dependency and control or inspect interactions.

Example:

```
mock_api.return_value = {'ok': True}
```

Logging

Definition: Logging records diagnostic and operational information about an application's execution.

Example:

```
import logging
logging.info('Started')
```

Debugger

Definition: A debugger lets a developer pause execution, inspect state, and step through code.

Example:

```
breakpoint()
```

Traceback

Definition: A traceback is Python's report showing the call stack and location of an unhandled exception.

Example:

```
ValueError: invalid literal
```

16. APIs, HTTP and FastAPI

API

Definition: An API is a defined interface through which software components communicate.

Example:

```
GET /users
```

REST

Definition: REST is an architectural style for networked systems that commonly uses HTTP resources and methods.

Example:

```
GET /users/1
```

HTTP

Definition: HTTP is an application-layer protocol used for communication between clients and servers.

Example:

```
GET /products
```

HTTP method

Definition: An HTTP method indicates the intended operation, such as GET, POST, PUT, PATCH, or DELETE.

Example:

```
POST /users
```

HTTP status code

Definition: A status code communicates the result of an HTTP request, such as 200, 201, 400, 404, or 500.

Example:

```
200 OK
```

Request

Definition: An HTTP request is a message sent by a client to a server, containing a method, target, headers, and optionally a body.

Example:

```
POST /login
```

Response

Definition: An HTTP response is the server's message containing a status code, headers, and optionally a body.

Example:

```
HTTP/1.1 200 OK
```

Endpoint

Definition: An endpoint is a specific API URL and method combination that performs an operation.

Example:

```
GET /users/{id}
```

Path parameter

Definition: A path parameter is variable data embedded in a URL path.

Example:

```
/users/{user_id}
```

Query parameter

Definition: A query parameter is optional key-value data supplied after ? in a URL.

Example:

```
/users?limit=10
```

Request body

Definition: The request body carries data sent to a server, commonly as JSON.

Example:

```
{"name": "Asha"}
```

FastAPI

Definition: FastAPI is a modern Python web framework for building APIs using Python type hints.

Example:

```
from fastapi import FastAPI
app = FastAPI()
```

Pydantic model

Definition: A Pydantic model defines structured data and performs validation and serialization.

Example:

```
class User(BaseModel):
    name: str
```

Dependency injection

Definition: Dependency injection supplies a component with the resources or services it needs instead of having it construct them itself.

Example:

```
Depends(get_db)
```

Middleware

Definition: Middleware is code that runs around request/response processing and can handle cross-cutting concerns.

Example:

```
@app.middleware('http')
```

CORS

Definition: Cross-Origin Resource Sharing is a browser security mechanism controlling which origins may access resources.

Example:

```
allow_origins=['https://example.com']
```

Authentication

Definition: Authentication verifies who a user or system is.

Example:

```
Bearer token
```

Authorization

Definition: Authorization determines what an authenticated user or system is allowed to do.

Example:

```
role == 'admin'
```

JWT

Definition: JSON Web Token is a compact signed token format commonly used to carry claims between parties.

Example:

```
Authorization: Bearer <token>
```

17. Databases and SQL Concepts

Database

Definition: A database is an organized system for storing and retrieving data.

Example:

```
users table
```

Relational database

Definition: A relational database stores structured data in tables related through keys.

Example:

```
PostgreSQL
```

SQL

Definition: SQL is a language used to define, query, and manipulate relational data.

Example:

```
SELECT * FROM users;
```

Primary key

Definition: A primary key uniquely identifies each row in a table.

Example:

```
id INTEGER PRIMARY KEY
```

Foreign key

Definition: A foreign key references a key in another table to represent a relationship.

Example:

```
user_id REFERENCES users(id)
```

Index

Definition: A database index is a data structure that can speed up lookups at the cost of storage and write overhead.

Example:

```
CREATE INDEX idx_email ON users(email);
```

Transaction

Definition: A transaction groups database operations into a unit that can be committed or rolled back.

Example:

```
BEGIN; UPDATE ...; COMMIT;
```

ORM

Definition: An Object-Relational Mapper lets application code work with database records through programming-language objects.

Example:

```
SQLAlchemy
```

Migration

Definition: A migration is a versioned change to a database schema.

Example:

```
alembic upgrade head
```

18. Async Python and Concurrency

Synchronous

Definition: Synchronous code generally waits for an operation to finish before moving to the next dependent operation.

Example:

```
result = read_file()
```

Asynchronous

Definition: Asynchronous programming allows tasks to pause while waiting for operations such as I/O, letting other work proceed.

Example:

```
result = await fetch_data()
```

Coroutine

Definition: A coroutine is an awaitable computation created by an `async def` function.

Example:

```
async def fetch(): ...
```

`async def`

Definition: Defines a coroutine function.

Example:

```
async def main(): pass
```

`await`

Definition: `await` pauses a coroutine until an awaitable completes.

Example:

```
data = await fetch()
```

Event loop

Definition: An event loop schedules and runs asynchronous tasks and handles I/O readiness.

Example:

```
asyncio.run(main())
```

Concurrency

Definition: Concurrency means multiple tasks can make progress during overlapping periods.

Example:

```
asyncio.gather(a(), b())
```

Parallelism

Definition: Parallelism means multiple computations execute at the same time, typically using multiple CPU cores or processors.

Example:

```
ProcessPoolExecutor
```

Thread

Definition: A thread is a unit of execution within a process that can share memory with other threads in that process.

Example:

```
threading.Thread(...)
```

Process

Definition: A process is an independent program execution environment with its own memory space.

Example:

```
multiprocessing.Process(...)
```

19. Memory and Python Internals

Reference

Definition: A reference is a name or container entry that points to an object.

Example:

```
a = [1,2]
```

Reference counting

Definition: CPython primarily tracks how many references point to an object and can reclaim objects when the count reaches zero.

Example:

```
del a
```

Garbage collector

Definition: Python's garbage collector helps detect and clean up reference cycles that reference counting alone cannot reclaim.

Example:

```
import gc
```

Garbage collection

Definition: Garbage collection is automatic reclamation of memory occupied by objects that are no longer reachable.

Example:

```
gc.collect()
```

GIL

Definition: In standard CPython builds, the Global Interpreter Lock historically prevents multiple threads from executing Python bytecode simultaneously within one interpreter.

Example:

```
threading
```

Shallow copy

Definition: A shallow copy creates a new outer container while retaining references to nested objects.

Example:

```
copy.copy(obj)
```

Deep copy

Definition: A deep copy recursively copies nested objects so the copied structure does not share those nested references.

Example:

```
copy.deepcopy(obj)
```

Hash table

Definition: A hash table stores key-value associations using hash values for efficient average-case lookup.

Example:

```
dict
```

20. Git and Software Development

Git

Definition: Git is a distributed version-control system used to track changes to source code.

Example:

```
git status
```

Repository

Definition: A repository is a Git-managed collection containing project history and files.

Example:

```
git init
```

Commit

Definition: A commit records a snapshot of staged changes in Git history.

Example:

```
git commit -m 'Add API'
```

Branch

Definition: A branch is a movable pointer to a line of development in Git.

Example:

```
git checkout -b feature/login
```

Merge

Definition: Merge combines histories from different branches.

Example:

```
git merge main
```

Pull request

Definition: A pull request is a platform workflow for proposing and reviewing changes before merging them.

Example:

```
Open PR on GitHub
```

CI/CD

Definition: Continuous integration and continuous delivery/deployment automate activities such as testing, building, and releasing software.

Example:

```
GitHub Actions
```

21. Security and Production Concepts

Environment variable

Definition: An environment variable is external configuration supplied to a process, commonly used for secrets and environment-specific settings.

Example:

```
DATABASE_URL=...
```

Secret

Definition: A secret is sensitive configuration such as an API key, password, or token that should be protected.

Example:

```
API_KEY
```

Input validation

Definition: Input validation checks that incoming data has the expected shape, type, range, and constraints.

Example:

```
age >= 0
```

SQL injection

Definition: SQL injection is a vulnerability where untrusted input changes the intended meaning of a database query.

Example:

```
Use parameterized queries.
```

Hashing

Definition: Hashing transforms data into a fixed-size digest; secure password storage uses password-hashing algorithms designed for this purpose.

Example:

```
bcrypt(password)
```

Encryption

Definition: Encryption transforms readable data into protected ciphertext using a key so authorized parties can decrypt it.

Example:

```
encrypt(data, key)
```

Rate limiting

Definition: Rate limiting restricts how frequently a client can make requests during a defined period.

Example:

```
100 requests/minute
```

Caching

Definition: Caching stores reusable results temporarily to reduce repeated computation or network/database work.

Example:

```
cache.get('user:1')
```

Environment separation

Definition: Development, testing, staging, and production environments are separated to reduce accidental impact and make deployment safer.

Example:

```
DEV / TEST / PROD
```

22. Interview-Important Definitions

PEP 8

Definition: PEP 8 is Python's style guide containing conventions for readable Python code.

Example:

```
snake_case for functions
```

PEP

Definition: A Python Enhancement Proposal is a document describing a proposed or accepted change, standard, or process related to Python.

Example:

```
PEP 8
```

EAFP

Definition: EAFP means 'easier to ask forgiveness than permission': code attempts an operation and handles exceptions when needed.

Example:

```
try: value = d[key]
except KeyError: ...
```

LBYL

Definition: LBYL means 'look before you leap': code checks conditions before attempting an operation.

Example:

```
if key in d: value = d[key]
```

Duck typing

Definition: Duck typing focuses on whether an object supports the required behavior rather than its exact declared type.

Example:

```
obj.read()
```

Dependency

Definition: A dependency is an external library, service, module, or component required by a program.

Example:

```
FastAPI depends on Starlette
```

Technical debt

Definition: Technical debt is the future cost created by choosing a quick or limited implementation instead of a more maintainable solution.

Example:

```
temporary workaround
```

Idempotent operation

Definition: An operation is idempotent if repeating it has the same intended effect as performing it once.

Example:

```
PUT /users/1
```

Serialization

Definition: Serialization converts an object or data structure into a format suitable for storage or transmission.

Example:

```
json.dumps(data)
```

Deserialization

Definition: Deserialization reconstructs usable data from a serialized representation.

Example:

```
json.loads(text)
```

API versioning

Definition: API versioning manages changes to an API contract so clients can continue to work predictably.

Example:

```
/api/v1/users
```

Scalability

Definition: Scalability is the ability of a system to handle increased workload by adding resources or improving architecture.

Example:

```
horizontal scaling
```

Maintainability

Definition: Maintainability is how easily software can be understood, modified, tested, and repaired.

Example:

```
clear modules + tests
```

Clean code

Definition: Clean code is code designed to be readable, understandable, testable, and maintainable.

Example:

```
small focused functions
```

Quick Revision Checklist

- Can I explain variables, objects, mutability, and dynamic typing?
- Can I use lists, tuples, sets, dictionaries, slicing, and comprehensions?
- Can I write functions using parameters, return, *args, and **kwargs?
- Can I explain LEGB, exceptions, context managers, and custom exceptions?
- Can I explain classes, inheritance, polymorphism, abstraction, and composition?
- Can I explain iterables, iterators, generators, decorators, and closures?
- Can I use type hints, dataclasses, and virtual environments?
- Can I build and test a REST API and explain HTTP status codes?
- Can I explain authentication vs authorization and basic API security?
- Can I work with SQL, databases, indexes, transactions, and an ORM?
- Can I explain async/await, concurrency, threads, processes, and the GIL?
- Can I use Git branches, commits, merges, and pull requests?
- Can I write tests, logs, configuration, and production-ready error handling?